Campus Event System — Complete SQL Query Reference Sheet

This reference document compiles every single SQL query used across the **Campus Event Registration & Login System** (EventFlow SQL) codebase, grouped logically by domain and feature set.

📁 1. Relational Database Schema (DDL)

These queries define the structure of the relational database. (Located in database/schema.sql).

1.1 Create Admins Table

```
CREATE TABLE admins (
    admin_id INT AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (admin_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

1.2 Create Students Table

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    department VARCHAR(50) NOT NULL,
    usn VARCHAR(20) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (student_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

1.3 Create Events Table

*Features **Foreign Key Constraint** linked to admins, setting created_by to NULL if an admin account is deleted.*

```
CREATE TABLE events (
    event_id INT AUTO_INCREMENT,
    title VARCHAR(150) NOT NULL,
    description TEXT NOT NULL,
    date DATE NOT NULL,
    time TIME NOT NULL,
    venue VARCHAR(100) NOT NULL,
    capacity INT NOT NULL,
    category VARCHAR(50) NOT NULL,
    created_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (event_id),
    FOREIGN KEY (created_by) REFERENCES admins(admin_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

1.4 Create Registrations Junction Table

*Features **Many-to-Many relationship**, composite unique constraints (prevents duplicate event signups), and **Cascade Delete**.*

```
CREATE TABLE registrations (
    registration_id INT AUTO_INCREMENT,
    student_id INT NOT NULL,
    event_id INT NOT NULL,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (registration_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (event_id) REFERENCES events(event_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT unique_registration UNIQUE (student_id, event_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

🔑 2. Student Authentication & Registration

(Located in controllers/authController.js).

2.1 Check if Email or USN is already Registered

```
SELECT student_id, email, usn
FROM students
WHERE email = ? OR usn = ?;
```

2.2 Create New Student Account

```
INSERT INTO students (full_name, email, password, department, usn)
VALUES (?, ?, ?, ?, ?);
```

2.3 Verify Student Login Credentials

```
SELECT * FROM students WHERE email = ?;
```

2.4 Verify Admin Login Credentials

```
SELECT * FROM admins WHERE email = ?;
```

🎓 3. Student Portal Operations

(Located in controllers/eventController.js).

3.1 Fetch Main Event Catalog

Calculates aggregate signup counters and checks if the active student has already registered.

```
SELECT e.event_id, e.title, e.description, e.date, e.time, e.venue, e.capacity, e.category,
       COUNT(r.registration_id) AS booked_count,
       (SELECT COUNT(*) FROM registrations WHERE event_id = e.event_id AND student_id = ?) AS reg
FROM events e
LEFT JOIN registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title, e.description, e.date, e.time, e.venue, e.capacity, e.category;
```

3.2 View Event details by ID

```
SELECT * FROM events WHERE event_id = ?;
```

3.3 Check Capacity & Double-Bookings before Joining

```
-- 1. Ensure event exists and fetch capacity
SELECT title, capacity FROM events WHERE event_id = ?;

-- 2. Check if student already signed up
SELECT registration_id FROM registrations WHERE student_id = ? AND event_id = ?;

-- 3. Calculate current booking count
SELECT COUNT(*) AS current_count FROM registrations WHERE event_id = ?;
```

3.4 Join Event (Register)

```
INSERT INTO registrations (student_id, event_id) VALUES (?, ?);
```

3.5 Cancel/Leave Event

```
DELETE FROM registrations WHERE student_id = ? AND event_id = ?;
```

3.6 Fetch Student Registration History

```
SELECT r.registration_id, e.event_id, e.title, e.date, e.time, e.venue, r.registered_at
FROM registrations r
INNER JOIN events e ON r.event_id = e.event_id
WHERE r.student_id = ?
ORDER BY r.registered_at DESC;
```

📊 4. Admin Portal Stats & CRUD Actions

(Located in controllers/adminController.js and controllers/eventController.js).

4.1 Overview Counters

```
-- Total Students
SELECT COUNT(*) AS total FROM students;

-- Total Events
SELECT COUNT(*) AS total FROM events;

-- Total Registrations
SELECT COUNT(*) AS total FROM registrations;
```

4.2 Chart Analysis: Top 5 Popular Events

```
SELECT e.title, COUNT(r.registration_id) AS count
FROM events e
LEFT JOIN registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY count DESC
LIMIT 5;
```

4.3 Chart Analysis: Student Department Distribution

```
SELECT department, COUNT(*) AS count
FROM students
GROUP BY department;
```

4.4 View Student Directory

```
SELECT student_id, full_name, usn, department, email, created_at
FROM students
ORDER BY created_at DESC;
```

4.5 Global Enrollment Logs (Composite Join)

```
SELECT r.registration_id, s.full_name, s.usn, s.department, e.title, e.venue, r.registered_at
FROM registrations r
INNER JOIN students s ON r.student_id = s.student_id
INNER JOIN events e ON r.event_id = e.event_id
ORDER BY r.registered_at DESC;
```

4.6 Create Campus Event

```
INSERT INTO events (title, description, date, time, venue, capacity, category, created_by)
VALUES (?, ?, ?, ?, ?, ?, ?, ?);
```

4.7 Update Event Details

```
UPDATE events
SET title = ?, description = ?, date = ?, time = ?, venue = ?, capacity = ?, category = ?
WHERE event_id = ?;
```

4.8 Delete Event

```
DELETE FROM events WHERE event_id = ?;
```

5. Preset Academic/Viva Queries (Live Compiler)

These specific, highly academic SQL queries demonstrate core relational concepts for project evaluations. (Located in controllers/queryController.js).

5.1 Multi-Table INNER JOIN

Retrieves all students signed up for a specific event.

```
SELECT s.student_id, s.full_name, s.usn, s.department, r.registered_at
FROM registrations r
INNER JOIN students s ON r.student_id = s.student_id
WHERE r.event_id = 1
ORDER BY s.full_name ASC;
```

5.2 LEFT JOIN & GROUP BY Aggregate

Shows all events, seat limits, and current active registration counts.

```
SELECT e.event_id, e.title, e.capacity, COUNT(r.registration_id) AS total_registrations
FROM events e
LEFT JOIN registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title, e.capacity
ORDER BY total_registrations DESC;
```

5.3 HAVING Clause Aggregate Filter

Filters grouped results to list only events with 2 or more signups.

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS total_regs
FROM events e
INNER JOIN registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
HAVING total_regs >= 2
ORDER BY total_regs DESC;
```

5.4 Subquery using `NOT IN`

Identifies students who have not registered for any events.

```
SELECT student_id, full_name, usn, department
FROM students
WHERE student_id NOT IN (
    SELECT DISTINCT student_id
    FROM registrations
);
```

5.5 Correlated Subquery

Identifies events that are fully booked (where registration count >= capacity).

```
SELECT event_id, title, capacity
FROM events e
WHERE e.capacity <= (
    SELECT COUNT(*)
    FROM registrations r
    WHERE r.event_id = e.event_id
);
```

5.6 Pattern Matching using `LIKE`

Retrieves students belonging to Computer Science.

```
SELECT student_id, full_name, usn, email, department
FROM students
WHERE department LIKE '%Computer Science%'
ORDER BY usn ASC;
```